

RELATÓRIO TÉCNICO FINAL: AMAZONIA.COM

Gabriel Pimentel Tabatinga

1. INTRODUÇÃO

O avanço das aplicações móveis e a necessidade de escalabilidade impulsionaram a adoção de sistemas distribuídos e arquiteturas baseadas em eventos. Este relatório documenta o desenvolvimento do ecossistema Amazonia.com, uma plataforma de logística e intermediação de entregas composta por dois aplicativos móveis distintos (Cliente e Prestador) integrados a um ecossistema de backend robusto.

O objetivo central do projeto foi aplicar conceitos avançados de Engenharia de Software, incluindo a separação de responsabilidades através da Clean Architecture, persistência de dados relacional e mensageria assíncrona orientada a eventos (Message-Oriented Middleware - MOM) para garantir o desacoplamento e a consistência eventual do sistema.

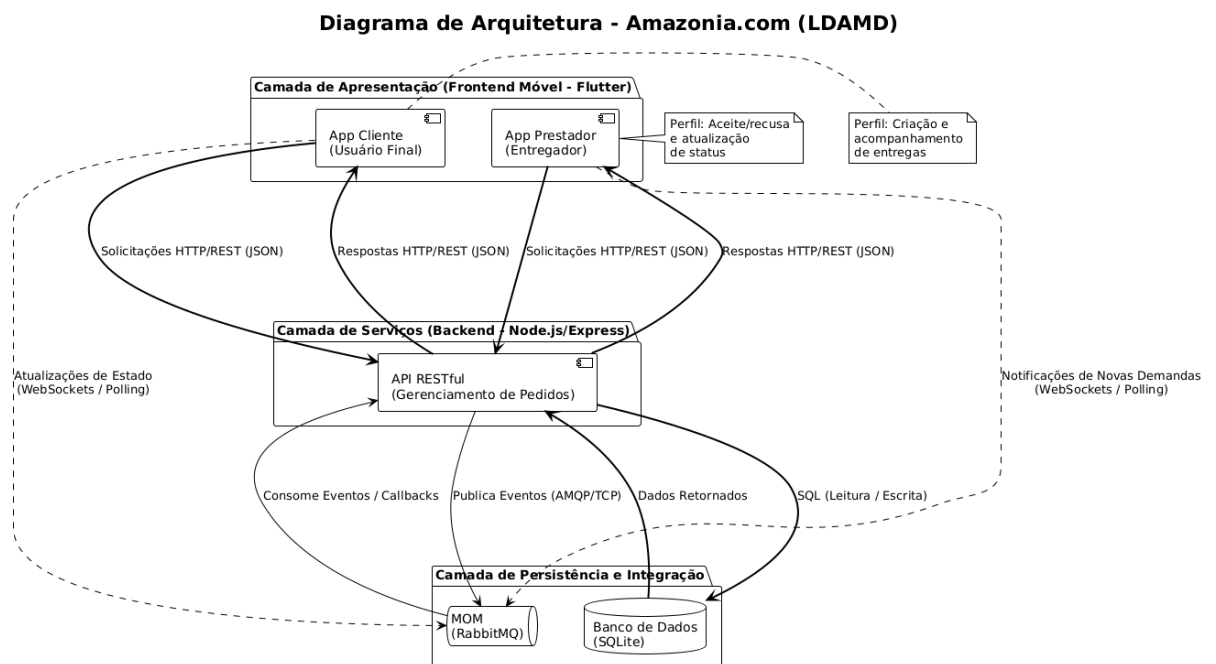
2. ARQUITETURA GERAL DO SISTEMA

O sistema adota uma abordagem cliente-servidor distribuída, dividida em três componentes independentes:

1. Backend REST API & Event Broker: Desenvolvido em Node.js com Express, atuando como o núcleo inteligente do sistema. Ele gerencia as regras de negócio, a segurança via tokens e a publicação de eventos no message broker.
2. Aplicativo do Cliente (amazonia_cliente): Interface em Flutter voltada para o consumidor final, permitindo a visualização de produtos, a criação de ordens de serviço e o cancelamento de pedidos pendentes.

3. Aplicativo do Entregador (amazonia_prestador): Interface em Flutter dedicada aos prestadores de serviço, permitindo a captura de demandas pendentes na fila, o gerenciamento de entregas em andamento e a conclusão dos fluxos de entrega.

Conforme ilustrado no diagrama de arquitetura, o fluxo de eventos lógicos (linhas pontilhadas) determina que as atualizações de estado e as notificações de novas demandas sejam enviadas aos aplicativos móveis a partir do processamento do MOM. Fisicamente, para manter o desacoplamento e a segurança da rede, optou-se por implementar essa comunicação através de um Polling Assíncrono gerenciado via HTTP, onde o Flutter consulta periodicamente a API RESTful, que por sua vez reflete a consistência eventual garantida pelo consumo assíncrono das filas do RabbitMQ.



3. DETALHAMENTO TÉCNICO E PADRÕES DE PROJETO

3.1. Organização do Código no Backend (Clean Architecture)

Para evitar o acoplamento excessivo e a criação de um código monolítico espaguete, o backend foi refatorado (em relação a sprint 1) seguindo os princípios de separação de conceitos (Separation of Concerns). A estrutura foi dividida em camadas bem definidas:

- Camada de Inicialização (server.js): Ponto de entrada isolado da aplicação, responsável por carregar variáveis de ambiente, inicializar conexões de infraestrutura e subir o servidor HTTP.
- Camada de Configuração (src/config/): Centraliza os acessos externos à infraestrutura (módulos isolados para inicialização do banco SQLite e conexões do RabbitMQ).
- Camada de Roteamento (src/routes/): Declara estritamente os endpoints da API, mapeando os verbos HTTP correspondentes para seus respectivos controladores.
- Camada de Controle (src/controllers/): Onde reside a lógica de aplicação. Intercepta as requisições, interage com a persistência e despacha eventos para o barramento de mensagens.

3.2. Segurança, Autenticação e Criptografia

A segurança da informação foi implementada através do JWT combinado com criptografia e hashing:

- Autenticação Baseada em Token: Ao realizar o login com sucesso, o servidor gera um token assinado digitalmente utilizando uma chave privada (JWT_SECRET). Esse token carrega o payload com o ID e o perfil do usuário (cliente ou entregador) com validade de 8 horas.
- Middleware de Proteção (authMiddleware): Atua como um interceptador de requisições nas rotas privadas. Ele extrai o token do cabeçalho HTTP Authorization: Bearer <token>, valida a assinatura e injeta os dados decodificados no objeto de requisição (req.userId), impedindo acessos não autorizados (Erros 403 Forbidden).

- **Criptografia de Senhas:** Para mitigar ataques de vazamento de dados, as senhas não são armazenadas em texto plano. Utilizou-se a biblioteca `bcryptjs` para gerar um hash adaptativo usando a técnica de salting com 10 rounds de processamento antes da inserção no banco SQLite.

3.3. Mensageria Assíncrona (Message-Oriented Middleware - MOM)

A comunicação assíncrona foi estruturada utilizando o RabbitMQ hospedado na nuvem via CloudAMQP, operando sob o protocolo AMQP. O desacoplamento do sistema baseia-se em duas filas principais:

- **fila_novos_pedidos:** Escuta os eventos de criação de pedidos. Quando um cliente solicita um produto, a API publica a mensagem e retorna imediatamente o status 201 ao cliente, enquanto o broker enfileira o processo para consumo assíncrono dos entregadores.
- **fila_atualizacao_status:** Gerencia a consistência de estados (pendente, aceito, concluído, cancelado), garantindo que as mudanças provocadas por um aplicativo reflitam nas notificações de logs e nos fluxos de controle internos do ecossistema.

4. DESENHO DO FRONTEND E ATUALIZAÇÃO ASSÍNCRONA

4.1. Arquitetura do App Flutter

Os dois aplicativos móveis foram estruturados sob uma variação da Clean Architecture para o ecossistema Dart:

- **Data Layer (Models & Services):** Contém os objetos de transferência de dados (DTOs) Produto e Pedido, capazes de serializar e desserializar estruturas JSON (`fromJson`). O `ApiService` centraliza o cliente HTTP, injetando dinamicamente o Token JWT recuperado do armazenamento local em cada requisição.

- Presentation Layer (Screens & Widgets): Telas de visualização construídas com componentes nativos do Flutter que interagem diretamente com o ciclo de vida do Framework (initState, dispose).

4.2. Persistência Local e Gerenciamento de Estado

- Cofre Local (shared_preferences): Utilizado para persistência persistente do token de autenticação no nível do sistema operacional (utilizando chaves UserDefaults no iOS e SharedPreferences no Android). Permite que o usuário permaneça conectado mesmo após o fechamento do app e viabiliza a operação de Logout limpando o registro.
- Sincronização por Polling Assíncrono: Para atender ao requisito de atualização assíncrona da interface sem a complexidade de sobrecarga de conexões persistentes WebSocket em canais mobile instáveis, implementou-se a estratégia de Polling gerenciado. O aplicativo realiza requisições HTTP automáticas ao backend a cada 5 segundos de forma silenciosa, atualizando o estado do componente (FutureBuilder) e redesenhando a interface caso o status do pedido mude no banco de dados.

5. CONCLUSÃO E AVALIAÇÃO DO SISTEMA

A separação do projeto em componentes independentes provou-se altamente eficaz. A modularização do backend eliminou redundâncias de código e facilitou a implementação do sistema de segurança baseado em tokens sem impactar a lógica de negócios pré-existente. No frontend, a criação de dois apps separados utilizando o mesmo núcleo de serviços (lib/data) validou a capacidade de reuso de código proporcionada por arquiteturas bem planejadas.

A inclusão do barramento RabbitMQ garante que o sistema possua alta resiliência, permitindo que requisições de entregas sejam registradas com sucesso mesmo sob picos de

tráfego, assegurando o cumprimento dos requisitos funcionais e não-funcionais estabelecidos para o projeto integrador.

6. REFERÊNCIAS BIBLIOGRÁFICAS

- MARTIN, Robert C. Arquitetura Limpa: O guia do artesão para estrutura e design de software. 1ª ed. Rio de Janeiro: Alta Books, 2018.
- TANENBAUM, Andrew S.; VAN STEEN, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2ª ed. São Paulo: Pearson, 2007.
- EXPRESS.js. Express - Node.js web application framework. Disponível em: <https://expressjs.com/>. Acesso em: jun. 2026.
- FLUTTER Dev. Flutter documentation: Architectural overview. Disponível em: <https://docs.flutter.dev/resources/architectural-overview>. Acesso em: jun. 2026.
- RABBITMQ. RabbitMQ Documentation - Tutorials and Guides. Disponível em: <https://www.rabbitmq.com/documentation.html>. Acesso em: jun. 2026.